[1]

Git and the Command Line

In this chapter, I will introduce what Git is and why we use it, and you'll make sure you have it installed on your computer. You will also set some Git configuration variables (aka *settings)*. I'll introduce the graphical user interface and the command line, which are the two tools you will use to interact with Git and the hands-on Rainbow project. To get you comfortable working in the command line, we will go over how to carry out some basic actions such as viewing the current directory location, navigating into and out of directories, and making directories. Finally, at the end of the chapter, you will prepare the text editor you will use to work on the Rainbow project in Chapter 2.

If you already have some experience working in the command line, then you may already know some of the information in this chapter. However, I don't recommend skipping the chapter because it prepares the setup you will use for the rest of the book.

[NOTE]

To understand how to use this book, you must have read "Using This Book" on page xii. If you have not read that section, I strongly recommend that you go back and do that now.

What Is Git?

Git is a technology that can be used to track changes to a project and to help multiple people to collaborate on a project. At a basic level, a project version controlled by Git

consists of a folder with files in it, and Git tracks the changes that are made to the files in the project. This allows you to save different versions of the work you're doing, which is why we call Git a *version control system*.

Git was created by Linus Torvalds to version control the work done on a large software development project called the Linux kernel. However, since Git can track changes to all sorts of files, it can be used for a wide variety of projects.

Git is a powerful technology, and the abundance of features it provides—as well as the fact that it was originally designed to be used in the command line—means that using it is a bit more complicated than just selecting File → Save on your computer.

To summarize, Git is a version control system that you can download onto your computer that allows you to track the history of a project and collaborate with other people. Next, let's take a look at Example Book Project 1-1 to see an example of how I might use Git for my Book project.

EXAMPLE BOOK PROJECT 1-1

Suppose I am writing a book, and I want to use Git to version control all the files in my Book project. Every time I make changes to the book, I can save a version of it using Git. For example, suppose I make changes to the book on Monday, Wednesday, and Friday, and I save one version on each of those days. This means I have at least three versions of my project. A version of a project in Git is called a *commit*. In Chapter 2, you will learn more about commits. For now, all you need to know is that in my example I have at least three commits.

These three commits allow me to look at the different versions of the book that I had at the end of my Monday work session, my Wednesday work session, and my Friday work session. Git also allows me to compare any of those commits (or saved versions of my project) to one another to check what changed between the different versions. This illustrates how Git helps me track the history of my project.

Now, suppose I decide to work on my Book project together with a coauthor. Git allows me and my coauthor to work on the same project at the same time and combine our work when we are ready. For example, I can work on chapter 1 and my coauthor can work on chapter 2, and when we're ready, we can combine the work we have done.

If we get an editor to review the book, they can also make edits to all the chapters of the book we have written, and we can integrate those changes into the main version of the book as well. This illustrates how Git is a useful tool for collaboration.

Next, let's learn about some of the other tools you will use in this learning experience and how you will be interacting with Git.

The Graphical User Interface and the Command Line

The two main ways to interact with a computer are by using the graphical user interface or the command line.

The *graphical user interface* (GUI) is the set of graphical representations of objects (icons, buttons, etc.) that allows you to interact with your computer. You can think of it as the point-and-click interface. For example, the folders represented by folder icons on your desktop are part of your computer's GUI.

The command line—also known as the command line interface (CLI), terminal, or shell—is a place where you can type text-based commands to interact with your computer.

The default way to work with Git is through the command line. However, there are also ways in which you can work with Git using a GUI: for example, by using a Git GUI client or a text editor that has Git integrations. This means you can carry out Git actions by clicking buttons and selecting options instead of entering commands in the command line.

In this book you will learn how to use Git in the command line, because this allows you to build a solid mental model of how it works and gives you access to all of its functionality. You will use your computer's GUI only for other actions, for example to look at files in the filesystem or to work in your text editor to manage your files. We'll take a closer look at the command line in the next section.

[NOTE]

Throughout this book, I will provide some specific instructions for macOS and Microsoft Windows users. If you are a Linux user, I will assume you already know some command line basics.

Opening a Command Line Window

To use the command line, you must open a command line window using a command line application. At any given point in time, in a command line window, you are in one particular directory, which we refer to as the *current directory*. A directory, for our purposes, is the same thing as a folder.

When you open a command line window there will be a command prompt in the upper-left corner. This is a short piece of text whose exact contents will differ depending on your operating system and computer settings. By default, however, the command prompt indicates the directory location in the command line (in other words, your current directory). When you open a new command line window, the directory location starts off at the current user directory (also known as the home folder), which is represented by the tilde sign (`~`). This directory location will be the only important part of the command prompt that you will have to identify for the exercises in this book. After the command prompt, there is a cursor that indicates where you are typing in the command line.

See Figure 1-1 for an annotated example of a generic command prompt. In the examples in this book, we use a dollar sign ($) at the end of the command prompt, but this is just one example of how a command prompt may end. Your command prompt may end with a different character or symbol.

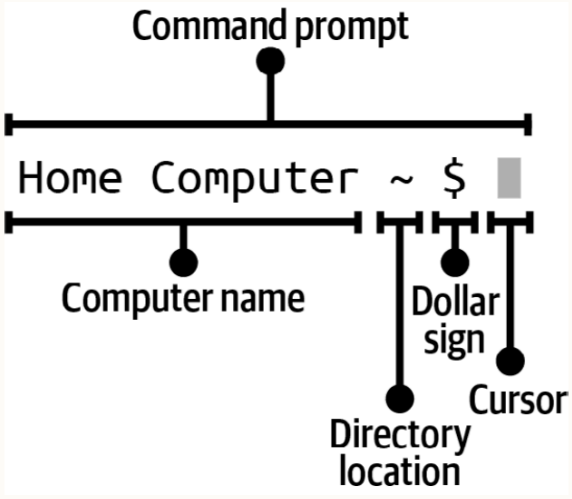


FIGURE 1-1

An example of a command prompt

The command line application you will use to complete the exercises in this book will depend on the operating system you are using:

- macOS—the command line application is called Terminal.
- Microsoft Windows—the command line application is called Git Bash. It will only be available if you have Git installed on your computer.

[NOTE]

If you are a Microsoft Windows user and you don't have Git installed, then you will need to go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*) and follow the steps to install Git for Microsoft Windows in order to have access to Git Bash before continuing with the rest of this chapter.

If you are a macOS user and you don't have Git installed, then you may continue with the rest of this section. You will install Git in "Installing Git" on page 7.

To open a command line window, you may use the search function on your computer to look up the command line application, select it, and open it. Go to Follow Along 1-1 to open a command line window and view the command prompt.

[FOLLOW ALONG 1-1]

1. Use your command line application to open a command line window.
2. Look at the command prompt in your command line window.

What to notice:

- The command prompt indicates the directory location.

Now that you have opened a command line window, let's cover how you will execute your first command in the command line.

Executing Commands in the Command Line

At the end of the command prompt in your command line window is the *cursor* that indicates where you will type commands. In the Terminal (macOS), by default, the cursor is on the same line as the command prompt, while in Git Bash (Microsoft Windows) it is on the line below. After you type a command, to execute it you have to press the Enter (Return) key.

If a step in a Follow Along section in this book includes a command in **bold** after a dollar sign ($), then you must execute it in the command line. If the command produces output, it will be shown below the command (not in bold). If the command is supposed to be executed in a directory other than the current user directory, the directory location will be indicated before the dollar sign.

Figure 1-2 shows an example of what it will look like when you execute a command in a Follow Along. In this case, I am executing the pwd command while in the desktop directory. You'll learn what the pwd command does later in this chapter; for now, take a look at Figure 1-2 to identify where it shows the directory location, the command to execute, and the output.

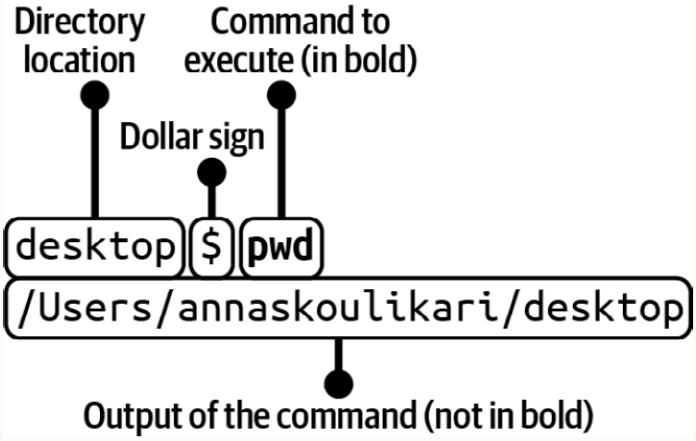


FIGURE 1-2

How to execute a command included in a Follow Along section

In the print version of this book, some longer commands are wrapped because of page width constraints. If you see a command line like the following:

rainbow $ **git remote add origin https://github.com/gitlearningjourney/rainbow-remote.git**

you should enter the entire command on one line, with a single space before the wrapped part (in this case, the URL). Reminders will be included in the appropriate places. If you're reading a version of this book that does not suffer from this line length constraint, you can ignore these reminders.

COMMAND OUTPUT

Some commands produce output, and some don't. For commands that produce output, I will provide sample output that is based on the Rainbow project I worked on in creating this book. This output was produced by the macOS operating system, but the output for Git-related commands does not differ between operating systems. In the few cases where non-Git-related command output differs between operating systems in a significant way, I will point it out in the text.

If, while carrying out the exercises in a Follow Along section, you see output that looks drastically different than the output in this book or you get an unexpected error, then you may have done something different than what the instructions indicate, and you may need to review the Follow Along steps.

[NOTE]

It is possible that Git will undergo updates in the future that may affect output in small ways. If I become aware of any significant changes, I will aim to document these situations on the errata page in the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

From now on, whenever you encounter a command in a Follow Along exercise, you should enter the command in bold in the command line and execute it.

EXECUTING THE FIRST COMMAND IN THE COMMAND LINE

The first command you will practice executing in the command line is the git version command. If you have Git installed on your computer, then it will provide the version number that you have installed. If you don't have Git installed on your computer, then the output will provide you with a message indicating that it is not installed.

To have access to all the commands used in the exercises in this book, I recommend you have a version of Git greater than 2.28. Go to Follow Along 1-2 to check if you have Git installed and what version it is.

[FOLLOW ALONG 1-2]

1. $ **git version**
   git version 2.35.1

What to notice:

- If you have Git installed, you see the version of it installed on your computer.
- If you don't have Git installed, you will instead see an error message.

If the git version command output indicates that you have a version of Git installed that is greater than 2.28, then you may skip the "Installing Git" section and go to the "Command Options and Arguments" section.

If the git version command output indicates you don't have Git installed on your computer or that the version of Git is older than 2.28, then you should continue on to the next section to install an up-to-date version of Git on your computer.

Installing Git

If you don't yet have Git (version 2.28 or greater) installed, go to Follow Along 1-3. Otherwise, go to the next section.

[FOLLOW ALONG 1-3]

1. Go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*) and follow the steps to download Git for your operating system.

Now that you've installed an up-to-date version of Git on your computer, let's continue learning a bit more about the commands you will be using in the Follow Along sections in this book.

Command Options and Arguments

Sometimes you will use commands with options and/or arguments. *Options* are settings that change the behavior of a command. An option follows a single dash (-) or a double dash (--).

Arguments are values that provide information to the command. They will be denoted by angle brackets (<>), indicating that these items should be replaced with user-supplied values. In the exercises, you will have to pass in a value for the argument, without including the angle brackets.

An example of a command with an option and an argument that you will use is git commit -m "<message>". In this example, -m is the option and <message> is the argument, as shown in Figure 1-3. We will cover what this command does in Chapter 3.

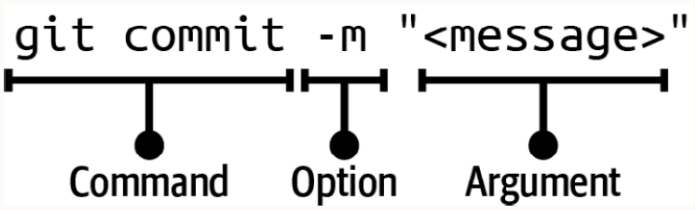


FIGURE 1-3

An example of a command with an option and an argument

In addition to entering commands in the command line, it is also important to learn how to clear them.

Clearing the Command Line

Every time you enter a command in the command line window, it will be listed directly under the previous command you entered (or its output). After you've entered a lot of commands, the command line window gets quite cluttered. To clear the contents of the command line window, you can use the clear command.

[SAVE THE COMMAND]

clear

Clear the command line window

Go to Follow Along 1-4 to practice using the clear command.

[FOLLOW ALONG 1-4]

```
1 $ clear
```

What to notice:

- The command line window is clear.

We've covered entering commands and clearing them from the command line window. Now it's time to prepare the next tool that will help you in your learning journey, the filesystem window.

Opening the Filesystem Window

You will use a filesystem application to open a filesystem window, which is part of the GUI. Throughout your learning journey, you will interact with both the filesystem window and the command line window. Therefore, it is useful to have both windows open side-by-side on your computer screen.

The filesystem application you will use will depend on your operating system:

- macOS—the filesystem application is Finder.
- Microsoft Windows—the filesystem application is File Explorer (or Windows Explorer).

Go to Follow Along 1-5 to open a filesystem window.

[FOLLOW ALONG 1-5]

1. Find your filesystem application and open up a filesystem window next to your command line window.

Now that you have both windows open, let's get back to some command line basics.

Working with Directories

As mentioned previously, at any given point in time in a command line window you are in one particular directory (the current directory). Assuming you haven't changed any default settings, when you first open the command line application you will start off in the current user directory (your home folder), indicated by the tilde sign (~) in the command prompt.

When you navigate to other directories in the command line, the command prompt will change to indicate the directory you are in. You can also use the pwd command, which stands for "print working directory," to see the path to the current directory.

[SAVE THE COMMAND]

pwd

Show the path to the current directory

Go to Follow Along 1-6 to practice using the pwd command.

[FOLLOW ALONG 1-6]

1. $ **pwd**
   /Users/annaskoulikari

What to notice:

- You are in your current user directory.

[NOTE]

In Follow Along 1-6, the output of the pwd command for Microsoft Windows users will be similar to /c/Users/annaskoulikari, which is slightly different from the output for macOS users. Keep this in mind when going through the rest of the exercises in this chapter.

The output of the pwd command prints the path to the current directory. In Follow Along 1-6, /Users/annaskoulikari is an example of a path. My name is Anna Skoulikari and annaskoulikari is my username on my computer. Users and annaskoulikari are two directories. Directories in a path are separated by a slash (/). The annaskoulikari directory is inside the Users directory.

Knowing the directory location in a command line window is useful because many commands show you information about or affect the current directory when you execute them. It also helps you with navigating through your filesystem, which we will cover later in this chapter.

Now that we have covered how to identify your current directory, let's explore how to view the actual contents of a directory.

VIEWING THE CONTENTS OF DIRECTORIES

You can view the contents of a directory in the GUI and the command line window. But before we get to that, I want to mention that there are two types of files and directories that exist in the filesystem: visible files and directories and hidden files and directories. *Visible* files and directories are always visible in the filesystem. *Hidden* files and directories are visible only in the filesystem if you change your settings to view them. They are often files or directories that store information that we, as users, don't need to access, such as application configurations and various system settings.

I do not recommend modifying or deleting hidden files or directories, unless you really know what you're doing. Once you change your settings to view hidden files and directories, they appear partially transparent (grayed out). Their names often start with a dot (.).

There are some important hidden files and directories that you will want to be aware of in your Git learning journey, so you need to know how to view them, both in the GUI and the command line.

In the GUI, to view hidden files and directories in a filesystem window you have to explicitly make them visible:

- macOS—to toggle between viewing and hiding hidden files and directories, press Cmd-Shift-dot.
- Microsoft Windows—you must alter the filesystem settings to view hidden files and directories. Consult online resources if necessary for step-by-step instructions for your computer.

Go to Follow Along 1-7 to view the hidden files and directories in your filesystem.

[FOLLOW ALONG 1-7]

1. Make hidden files and directories visible in your filesystem.

In the command line, to view a list of the visible files and directories in the current directory you use the ls command (which stands for "list").

To view both visible and hidden files and directories in the current directory you use the ls command with the -a option: ls -a.

[SAVE THE COMMAND]

ls

List visible files and directories

ls -a

List hidden and visible files and directories

Go to Follow Along 1-8 to practice using these commands to list different kinds of files.

[FOLLOW ALONG 1-8]

1 **$ ls**

```
Applications            Downloads               Music
Desktop                 Library                 Pictures
Documents               Movies                  Public
```

2 **$ ls -a**

```
.                       .config                 Desktop
..                      .local                  Documents
..CFUserTextEncoding    .npm                    Downloads
.DS_Store               .ssh                    Library
.Trash                  .viminfo                Movies
.bash_history           .vscode                 Music
.bash_sessions          .yarnrc                 Pictures
.bashrc                 Applications            Public
```

What to notice:

- The names of many hidden files and directories start with a dot (.).

- The visible and hidden files and directories shown in the output in this book will be different from the ones on your computer because the contents of everyone's computers are different.

Now that we have covered how to identify the current directory and view its contents in the command line, let's explore how to move between directories.

NAVIGATING INTO AND OUT OF A DIRECTORY

In the GUI, to go into a directory you can double-click on it. In the command line, to go into a directory you use the cd command, which stands for "change directory," and pass in either the name of the directory or the path to the directory.

[SAVE THE COMMAND]

cd <path_to_directory>

Change directory

Go to Follow Along 1-9 to practice by navigating into the desktop directory.

[FOLLOW ALONG 1-9]

1. $ **cd desktop**
2. desktop $ **pwd**
   /Users/annaskoulikari/desktop

What to notice:

- In step 1, the cd command does not produce any output.
- In step 2, the command prompt and the pwd command output indicate that you are in the desktop directory.

Earlier in this chapter, I mentioned that the command prompt shows the directory location. In Follow Along 1-9, notice that the command prompt updates to show that your

current directory is the desktop directory. By default, the way the directory location is presented differs depending on the operating system you use:

- macOS—in the Terminal, the name of the current directory is displayed.
- Microsoft Windows—in Git Bash, the path to the current directory is displayed.

[NOTE]

Navigating into and out of directories in the command line does not affect what you are viewing in the filesystem. For example, navigating into the desktop directory in the command line will not automatically cause your filesystem to display the contents of the desktop directory.

In the GUI, to go back to the parent directory you can select the back button. In the command line, to go back to the parent directory you can pass in two dots (..) to the cd command. Two dots represents the parent directory of the current directory. Go to Follow Along 1-10 to try this out.

[FOLLOW ALONG 1-10]

1. desktop $ **cd ..**
2. $ **pwd**
   /Users/annaskoulikari

What to notice:

- In step 2, the command prompt and the pwd output indicate that you are back in the current user directory.

Now that we have covered how to navigate into and out of directories, let's go over how to create a new one.

CREATING A DIRECTORY

In the GUI, you can create a directory by right-clicking or selecting the relevant menu option or icon. In the command line, to create a directory you will use a command called mkdir, which stands for "make directory." The directory will be created inside the current directory when you execute the command.

[SAVE THE COMMAND]

mkdir <directory_name>

Create a directory

To keep things simple, avoid including spaces in your directory names. If a directory name contains spaces, then you have to make modifications to how you use certain commands in the command line, which makes your tasks more complicated.

[NOTE]

In general, when working in the command line you should avoid using spaces in the names of any files, directories, or other things you create because it can cause complications when you use commands.

As mentioned in the Preface, throughout this book you will work on one project in which you will create and edit files to list the colors of the rainbow, as well as some colors that are not part of the rainbow. This is not a realistic example of a project typically version controlled with Git; it's a simplified example that will allow you to focus on learning how Git works.

Since the main objective of the sample project is to list the colors of the rainbow, you will give your project directory the name rainbow. You will create this project directory in the desktop directory so that you can easily see it from the desktop on your computer screen. Go to Follow Along 1-11 to create your directory.

[FOLLOW ALONG 1-11]

```
1 $ cd desktop

2 desktop $ mkdir rainbow

3 desktop $ ls
  rainbow
```

What to notice:

- In step 3, the ls output will show the rainbow project directory you just created along with any other directories or files you have in your desktop directory. These are not displayed in the sample output, as the contents on your computer will differ from mine.

If you look on the desktop of your computer, you should see the rainbow project directory you just created, as shown in Figure 1-4.

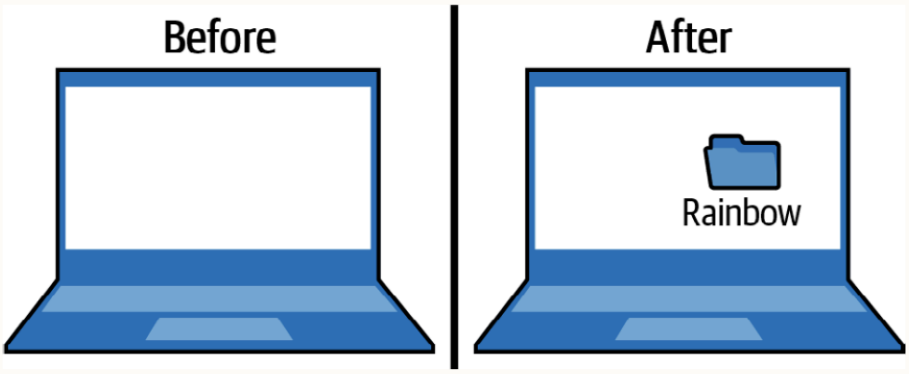


FIGURE 1-4

An example of your desktop before and after you create the rainbow project directory

Next, you'll notice that just because you created the rainbow project directory, this does not mean you navigated into it in the command line. Go to Follow Along 1-12 to explicitly navigate into the rainbow project directory in the command line.

[FOLLOW ALONG 1-12]

1. desktop $ **cd rainbow**

2. rainbow $ **pwd**
   /Users/annaskoulikari/desktop/rainbow

What to notice:

- In step 2, the command prompt and the output from the pwd command indicate you are in the rainbow directory.

You have created and navigated into the rainbow directory. Now, what happens if you close the command line window and open it back up again?

Closing the Command Line

By default, if you close the command line window and then open it up again, the directory location will reset to the current user directory. Therefore, you will have to navigate to the directory you want to work in again. Go to Follow Along 1-13 to test this out.

[FOLLOW ALONG 1-13]

1. rainbow $ **pwd**
   /Users/annaskoulikari/desktop/rainbow
2. Close your command line window and then open a new command line window.
3. $ **pwd**
   /Users/annaskoulikari
4. $ **cd desktop**
5. desktop $ **cd rainbow**
6. rainbow $ **pwd**
   /Users/annaskoulikari/desktop/rainbow

At this point, we have covered several command line basics. Next, you will set some basic Git configurations.

Setting Git Configurations

Git configurations are settings that allow you to customize how Git works. They consist of variables and their values, and they are stored in a couple of different files. To work with Git, you must set a few configuration variables related to user settings.

Before setting any variables, you'll check if a global Git configuration file exists in your filesystem and, if so, which variables have been set. To do so you will use the git config command, passing in the --global and --list options.

Note that the git config command with the --global option is an example of a command where it does not matter what your current directory is when you execute it; it will only ever show information about or change information in the global Git configuration file. This is a hidden file called .gitconfig that is usually created in the current user directory.

[SAVE THE COMMAND]

git config --global --list

List the variables in the global Git configuration file and their values

Go to Follow Along 1-14 to try this out.

[FOLLOW ALONG 1-14]

1. $rainbow **git config --global --list**
   fatal: unable to read config file '/Users/annaskoulikari/.gitconfig': No such file or directory

What to notice:

- The output shown here indicates what you will see if you have never set any variables in your global Git configuration file. In this case, you will get an error indicating that the file does not exist. If you have set variables in your global Git configuration file, then your output will display the variables you have set and their values.

The two variables we are interested in for this book are user.name and user.email. Every time someone saves a version of a project (or in other words, makes a commit), Git will note the name and email address of the individual and associate it with that saved version. The user.name and user.email variables are used to set the name and email address that will be saved for the commits you make. This means that you can see who worked on what in a Git project. Configuring these variables is required in order to work with Git. Keep in mind that anyone who is able to see a list of your commits in any project will be able to view your email address, so make sure to use an address you don't mind other people viewing.

To set these variables in your global Git configuration file, you pass them as arguments to the git config command, entering your desired values inside quotation marks (remember, you should not include the angle brackets around the values).

[SAVE THE COMMAND]

git config --global user.name "<name>"

Set your name in the global Git configuration file

git config --global user.email "<email>"

Set your email address in the global Git configuration file

If in the output of Follow Along 1-14 these variables are set to the values you want them to be, then you may skip Follow Along 1-15. If these variables do not appear or are not set to the values you want them to be, decide what name and email address you want to associate with the work you do on the Rainbow project, and go to Follow Along 1-15. Be sure to swap in the values you want to use in place of my username and email address.

[FOLLOW ALONG 1-15]

1. rainbow $ **git config --global user.name "annaskoulikari"**
2. rainbow $ **git config --global user.email "gitlearningjourney@gmail.com"**
3. rainbow $ **git config --global --list**
   user.name=annaskoulikari
   user.email=gitlearningjourney@gmail.com

What to notice:

- In step 3, in the git config --global --list output, the user.name and user.email variables are set to the values you entered.

Now that you've installed Git and prepared your user settings, the final tool you need to work on a Git project is a text editor.

Preparing a Text Editor

A Git project consists of files and directories that are version controlled. Git can version control all kinds of file types. In the Rainbow project, you will use a text editor to work with simple text files (that have an extension of .txt).

A *text editor* is a program that allows a user to edit plain text. You will need a text editor to carry out many of the Follow Along exercises in this book. A text editor is different from a word processor, which is mainly used to edit rich text. Examples of word processors are Microsoft Word and Google Docs; these cannot be used to manage the files of a Git project. *Rich text* is text with styles attached to or embedded within it. If you can see that text is bold or italicized, then that is rich text.

Some text editors make it a lot easier to work on Git projects than others. This book is written in a way that allows you to use whichever text editor you prefer. If you already have a text editor installed that you have used to work on Git projects, then feel free to use that for the exercises in this book. If you're not sure which text editor to use, I highly recommend Visual Studio Code (*https://code.visualstudio.com*): it's a popular text editor, and it is the one I used when working on the Rainbow project in this book. Go to Follow Along 1-16 to prepare a text editor.

[FOLLOW ALONG 1-16]

1. Choose your preferred text editor. If you don't already have one installed, then download a text editor.

2. Open the rainbow project directory in a text editor window.

Integrated Terminals

Some advanced text editors (also referred to as *integrated development environments*, or IDEs), such as Visual Studio Code, include a version of a command line within them, usually referred to as an *integrated terminal*, in which you can execute the commands you would normally use in your command line window.

When you use an integrated terminal it may be easier to manage your screen space, because the terminal is already part of the text editor window. However, this depends on personal preference. If your text editor has an integrated terminal, you may choose to use that instead of a separate command line window to execute the Git commands in the rest of the exercises in this book. Either will work just fine, so it is up to you to decide what you prefer.

In the rest of this book, whenever I want to refer to the place where you need to execute commands I will refer to the command line window; however, keep in mind that this also includes the integrated terminal.

Now, with Git installed and your text editor ready to go, you are ready to start working on the Rainbow project!

Summary

In this chapter, you learned a little about Git and saw that it is a useful tool for tracking the history of a project and collaborating with others. You prepared to work on a project using Git by installing an up-to-date version on your computer and setting some basic Git configuration variables.

You also learned some command line basics, such as how to view the contents of directories, how to navigate into and out of them, and how to make them in the first place. Along the way, you created the project directory that you will be using throughout this learning journey, called rainbow.

Finally, you prepared a text editor so that you can create and edit files for any Git project you want to work on.

Now you're ready to move on to Chapter 2, where you will actually turn your rainbow project directory into a Git repository and start learning about the most important areas when working with Git.

Previous chapter

< [Preface]

Next chapter

Chapter 2. Local Repositories >